Benchmarking Two-Stage, One-Stage, and Transformer Object Detectors on Under-Sampled Marine Biodiversity Imagery

A Deep Learning Project Report

Student:	Charan Rasineni
Program:	B.Tech, Computer Science and Engineering
School:	School of Computer Science and Engineering (SCOPE)
University:	VIT-AP University, Amaravati, 522241, India
Email:	charan.24bce7827@vitapstudent.ac.in

Mentor:	Dr. Rajesh Duvvuru
Role:	Mentor, SCOPE, VIT-AP University
Email:	rajesh.duvvuru@vitap.ac.in

November 4, 2026

Abstract

Automated monitoring of marine biodiversity from remotely operated vehicle (ROV) and autonomous underwater vehicle (AUV) imagery requires object detectors capable of identifying dozens of poorly sampled taxonomic groups under challenging optical conditions. This report presents a controlled benchmark of three representative object detection architectures — the two-stage Faster R-CNN R50-FPN, the one-stage YOLOv8s, and the end-to-end transformer-based RT-DETR-L — on the FathomNet CLEF 2026 underwater object detection dataset, a community benchmark comprising 5,817 labelled training images and 646 validation images distributed across 32 taxonomic classes with a strongly long-tailed frequency distribution.

All three detectors are trained and evaluated under an identical experimental protocol: the same stratified 90/10 image-level split (seed 42), the same 640×640 input resolution, the same COCO-style mean average precision (mAP) evaluation via `pycocotools`, and the same confidence threshold (0.001) and per-image detection cap (300). Experiments are conducted on a single NVIDIA T4 GPU with 15 GiB memory.

Faster R-CNN R50-FPN achieves the strongest overall accuracy ($\text{mAP}_{50\text{-}95} = 0.333$, $\text{mAP}_{50} = 0.483$), followed by YOLOv8s (0.313/0.428) and RT-DETR-L (0.231/0.323). YOLOv8s is the fastest detector at 24.0 ms per image, versus 46.4 ms for Faster R-CNN and 62.2 ms for RT-DETR-L. A consistent head-to-tail degradation is observed across all three architectures, reflecting the long-tailed class distribution, and small objects are effectively undetected at the nominal input resolution. These findings provide a practical accuracy–cost–latency trade-off for deploying underwater perception pipelines on ROV/AUV platforms.

Keywords: Object detection; Faster R-CNN; YOLOv8; RT-DETR; underwater imagery; marine biodiversity; FathomNet; long-tailed learning; benchmarking; deep learning.

Contents

1 Introduction **3**

2 Related Work **4**
2.1 Underwater Object Detection 4
2.2 Detector Families 4
2.3 Long-Tailed Recognition 4

3 The FathomNet CLEF 2026 Dataset **4**
3.1 Scope and Collection 4
3.2 Dataset Statistics 5
3.3 Image and Object Geometry 6
3.4 Split Protocol 6

4 Methods **6**
4.1 Detector Architectures 6
4.1.1 Faster R-CNN R50-FPN (Two-Stage) 6
4.1.2 YOLOv8s (One-Stage) 6
4.1.3 RT-DETR-L (Transformer-Based) 7
4.2 Experimental Protocol 7
4.3 Evaluation Metrics 7
4.4 Inference Cost Measurement 7

5 Results **8**
5.1 Overall Performance 8
5.2 Training Curves 8
5.3 Per-Class Analysis 9
5.4 Object-Size Analysis 9

5.5 Inference Cost 10
5.6 Qualitative Results 10
5.7 Grad-CAM Analysis 10

6 Discussion **12**

7 Limitations **12**

8 Conclusion and Future Work **12**

A Full Per-Class AP Table **14**

B Head/Mid/Tail and Size AP by Detector **14**

C Hardware and Software **15**

1 Introduction

The oceans cover more than 70% of Earth's surface and host a vast fraction of the planet's biodiversity, yet large portions of the deep sea remain poorly explored and documented. Remotely operated vehicles (ROVs) and autonomous underwater vehicles (AUVs) equipped with cameras now routinely collect terabytes of underwater imagery, offering an unprecedented window into marine ecosystems. However, the sheer volume of collected imagery far exceeds the capacity of human experts to annotate and identify organisms manually. Automated object detection — the task of localizing and classifying instances of interest in images — is therefore a critical enabling technology for scalable marine biodiversity monitoring.

Underwater object detection is substantially more difficult than terrestrial object detection. Water absorbs and scatters light, causing reduced contrast, colour cast, and limited visibility, particularly at depth. Turbidity, variable illumination, backscatter, occlusion, and biological camouflage further degrade image quality. Many marine taxa are small, rare, or exhibit high intra-class morphological variation, and annotated training data are scarce for most species. These challenges are compounded by extreme class imbalance: a small number of common taxa account for the majority of annotated instances, while numerous rare taxa are represented by only a handful of examples. A detector deployed in this setting must therefore generalize across both frequent "head" classes and a long "tail" of under-sampled taxa.

Recent years have seen rapid progress in object detection, with three dominant architectural families emerging: two-stage detectors (exemplified by Faster R-CNN [6] and its Feature Pyramid Network [7] extension), one-stage dense detectors (exemplified by YOLO [9]), and end-to-end transformer-based detectors (exemplified by DETR [11] and RT-DETR [13]). Each family makes different inductive biases and computational trade-offs, but their relative performance on long-tailed underwater imagery has not been systematically characterized under a controlled protocol. This report addresses that gap. We benchmark Faster R-CNN R50-FPN, YOLOv8s, and RT-DETR-L on the FathomNet CLEF 2026 dataset, a benchmark for underwater object detection drawn from the FathomNet image database [1] and the ImageCLEF LifeCLEF 2026 track [2]. The dataset contains 5,817 labelled training images and 646 validation images spanning 32 taxonomic classes, with a strongly long-tailed annotation distribution in which the three most frequent classes account for 53.4% of training annotations while the rarest class has only six instances. All models are trained and evaluated under an identical protocol to isolate architectural differences from confounds such as data split, input resolution, and evaluation settings.

The contributions of this work are:

1. A controlled, equal-protocol benchmark of a two-stage, one-stage, and transformer detector on FathomNet CLEF 2026 underwater imagery.

2. A head/mid/tail and object-size stratified error analysis revealing where each architecture succeeds and fails.

3. An accuracy–inference-cost–latency trade-off analysis relevant to deployment on resource-constrained ROV/AUV platforms.

4. A documented assessment of GPU memory constraints for transformer detectors on commodity hardware (15 GiB T4).

The remainder of this report is organized as follows. Section 2 reviews related work. Section 3 describes the dataset and its properties. Section 4 details the detector architectures, experimental protocol, and training setup. Section 5 presents quantitative and qualitative results. Section 6 discusses the findings, Section 7 outlines limitations, and Section 8 concludes with future work.

2 Related Work

2.1 Underwater Object Detection

Underwater vision differs fundamentally from terrestrial computer vision. Light attenuation in water is wavelength-dependent, with red wavelengths absorbed first, yielding a characteristic blue-green colour cast; scattering produces backscatter and reduces contrast; and illumination is often non-uniform due to artificial lighting on ROVs/AUVs. Chen et al. [14] survey underwater object detection in the era of artificial intelligence, identifying turbidity, low contrast, colour distortion, small objects, and class imbalance as the principal challenges. Earlier work on the FathomNet training database demonstrated that large-scale annotated underwater image collections can support the training of accurate deep detectors [3]. The FathomNet CLEF 2026 benchmark extends this effort by providing a standardized, community-maintained testbed for underwater object detection [2].

2.2 Detector Families

Two-stage detectors. Faster R-CNN [6] introduced the Region Proposal Network (RPN), which shares convolutional features between proposal generation and detection, enabling near real-time two-stage detection. The Feature Pyramid Network (FPN) [7] augments this architecture with a top-down pathway and lateral connections, producing strong feature representations at multiple scales — particularly valuable for small objects. Faster R-CNN with an R50-FPN backbone is a widely used accuracy benchmark in object detection.

One-stage detectors. The You Only Look Once (YOLO) family reformulates detection as a single regression problem, predicting bounding boxes and class probabilities directly from dense anchor points (or anchor-free keypoints in recent versions). YOLOv8 [9], distributed by Ultralytics [10], balances speed and accuracy through an anchor-free detection head, a CSPDarknet backbone, and efficient training augmentations. The "s" (small) variant, YOLOv8s, is designed for resource-constrained deployment while retaining competitive accuracy.

Transformer-based detectors. DEtection TRansformer (DETR) [11] treats object detection as a set prediction problem, eliminating hand-designed components such as NMS and anchor generation. Deformable DETR [12] improves convergence and small-object handling via deformable attention. RT-DETR [13] is an end-to-end real-time transformer detector that interleaves CNN backbone features with an efficient hybrid encoder and IoU-aware query selection, aiming to combine transformer accuracy with real-time inference. RT-DETR-L denotes the large variant evaluated here.

2.3 Long-Tailed Recognition

Long-tailed distributions are common in biodiversity datasets, where a few species dominate observations and most species are rare. Standard cross-entropy training tends to bias detectors toward head classes at the expense of tail classes. Although this report does not propose a new long-tailed algorithm, the head/mid/tail analysis in Section 5.3 quantifies how the architecture-independent data distribution constrains performance across all three detector families.

3 The FathomNet CLEF 2026 Dataset

3.1 Scope and Collection

The FathomNet CLEF 2026 dataset is an underwater object detection benchmark built on the FathomNet global image database [1, 3] and organized as the ImageCLEF LifeCLEF 2026 underwater object detection track [2]. Imagery is collected from ROV and AUV platforms across diverse marine environments. Each image is annotated with bounding boxes and taxonomic

labels by domain experts, yielding a multi-class detection benchmark with 32 taxonomic classes spanning invertebrates, fishes, corals, and other marine organisms.

3.2 Dataset Statistics

Table 1 summarizes the dataset. The training set comprises 5,817 labelled images and the validation set 646 images, distributed over 32 classes. The annotation distribution is strongly long-tailed: the three most frequent classes (bony fish, brittle star, urchin — the "head" group) account for approximately 53.4% of all training annotations, while the rarest class contains only six annotated instances. The remaining classes are organized into a "mid" group (anemone, sea fan, sponge, stony coral) and a "tail" group comprising the other 25 classes.

Table 1: FathomNet CLEF 2026 dataset statistics.

Property	Value
Training images	5,817
Validation images	646
Number of classes	32
Annotation distribution	Long-tailed
Head classes (top 3)	bony fish, brittle star, urchin
Mid classes	anemone, sea fan, sponge, stony coral
Tail classes	25 remaining classes
Top-3 share of annotations	$\approx 53.4\%$
Rarest class annotation count	6

Figure 1 visualizes the per-class annotation counts, illustrating the severity of the long tail. The imbalance has direct consequences for detector training: head classes receive dense gradient signal, whereas tail classes are under-represented and prone to being missed entirely (several tail classes achieve zero average precision across all three detectors, as shown in Section 5.3).

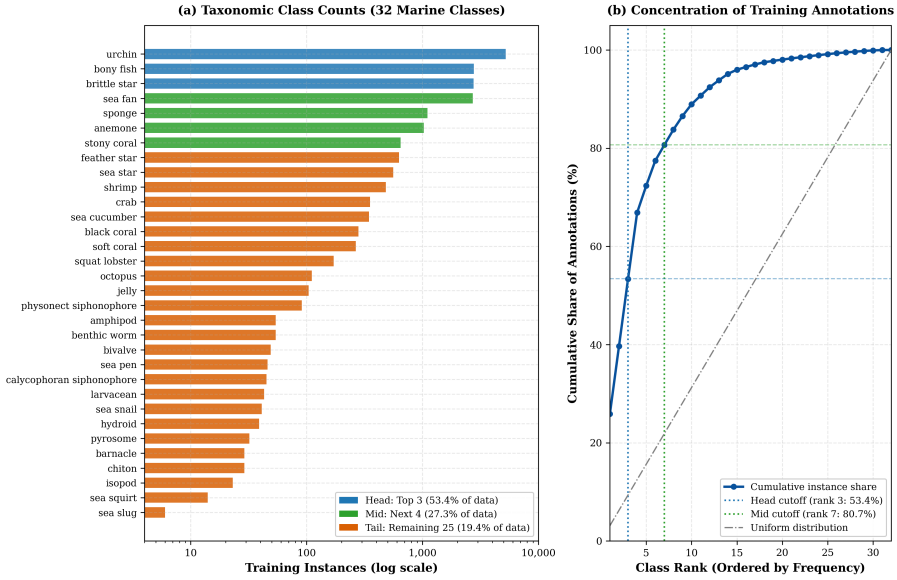


Figure 1: Per-class annotation counts in FathomNet CLEF 2026, illustrating the long-tailed distribution. Head classes (bony fish, brittle star, urchin) dominate the annotation count, while most tail classes are represented by only a few dozen instances.

3.3 Image and Object Geometry

Figure 2 summarizes image resolution and object-size statistics. Input images are resized to 640×640 pixels for training and evaluation, a common compromise that fits within GPU memory while preserving sufficient detail for medium and large objects. However, many marine organisms — particularly small invertebrates such as amphipods, isopods, and benthic worms — occupy a tiny fraction of the image area after resizing. As shown in Section 5.4, small objects are effectively undetected at this resolution by all three architectures.

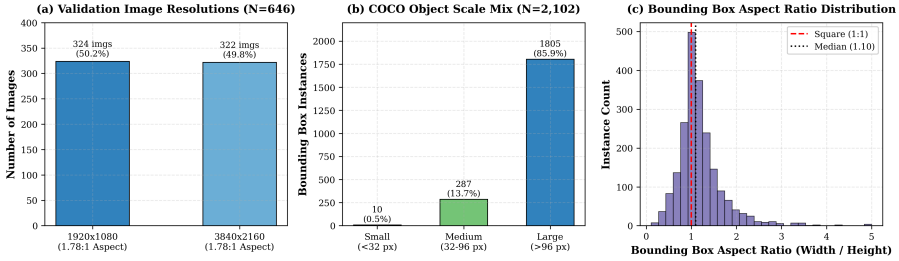


Figure 2: Image and object geometry statistics for FathomNet CLEF 2026, including resolution distribution and bounding-box area percentiles.

3.4 Split Protocol

To ensure a fair comparison, all three detectors are trained and evaluated on the same stratified 90/10 image-level train/validation split with random seed 42. Stratification preserves the class distribution across splits, and the split indices are persisted so that every model sees exactly the same data. Evaluation uses COCO-style bounding-box average precision via `pycocotools` [5], with mAP computed over IoU thresholds $[0.50 : 0.05 : 0.95]$ (denoted $\text{mAP}_{50\text{-}95}$) and at IoU 0.50 ($\text{mAP}_{50}$). The confidence threshold is fixed at 0.001 and the per-image detection cap at 300 for all models.

4 Methods

4.1 Detector Architectures

4.1.1 Faster R-CNN R50-FPN (Two-Stage)

Faster R-CNN [6] with a ResNet-50 backbone and Feature Pyramid Network neck [7] is the canonical two-stage detector. In the first stage, a Region Proposal Network proposes candidate object locations; in the second stage, RoIAlign classifies each proposal and refines its bounding box. The two-stage design localizes objects at coarse granularity first, then refines, which often yields superior accuracy on small and cluttered objects at the cost of higher computation and latency. This implementation uses the torchvision/Mask R-CNN lineage [8] and is trained in single precision with batch size 2 on a 15 GiB T4 GPU.

4.1.2 YOLOv8s (One-Stage)

YOLOv8s [9] is the small variant of the YOLOv8 one-stage detector family, distributed through the Ultralytics framework [10]. It uses an anchor-free detection head, a CSPDarknet-based backbone, and predicts boxes and class probabilities directly from feature maps. One-stage detectors avoid the two-stage propose-then-classify pipeline, making them substantially faster and lighterweight. YOLOv8s has roughly a quarter of the parameters of Faster R-CNN and is mixed-precision trained with batch size 4.

4.1.3 RT-DETR-L (Transformer-Based)

RT-DETR [13] is an end-to-end transformer-based real-time object detector. Building on DETR [11] and Deformable DETR [12], RT-DETR uses a hybrid efficient encoder and IoU-aware query selection to achieve strong accuracy without NMS. The "L" (large) variant evaluated here has the highest parameter count and memory footprint of the three models, requiring mixed-precision training with batch size 4 to fit on the T4 GPU.

4.2 Experimental Protocol

All models share the following protocol:

- **Data:** identical stratified 90/10 split, seed 42 (§3).
- **Input size:** 640 × 640 pixels.
- **Epochs:** 30 training epochs per model.
- **Evaluation:** COCO mAP via `pycocotools`, confidence threshold 0.001, max detections 300 per image.
- **Hardware:** NVIDIA T4 GPU, 15 GiB memory.

Framework-specific differences (resizing rules, optimizers, augmentation, checkpoint selection) remain, so the results compare representative default training setups rather than strictly equal compute. Training configurations are summarized in Table 2.

Table 2: Training configurations per detector.

Configuration	Faster R-CNN	YOLOv8s	RT-DETR-L
Family	Two-stage	One-stage	Transformer
Backbone	R50-FPN	CSPDarknet	Hybrid encoder
Batch size	2	4	4
Precision	FP32	Mixed (AMP)	Mixed (AMP)
Epochs	30	30	30
Input size	640 px	640 px	640 px
GPU memory (approx.)	High	Moderate	Highest

4.3 Evaluation Metrics

The primary metric is COCO mean average precision. $\text{mAP}_{50\text{-}95}$ averages AP over IoU thresholds from 0.50 to 0.95 in steps of 0.05, rewarding both localization and classification quality. mAP_{50} uses a single IoU threshold of 0.50 and is more forgiving of localization error. Additional analyses examine per-class AP, head/mid/tail grouping, object-size stratified AP (small/medium/large), and inference latency.

4.4 Inference Cost Measurement

Latency is measured on the training hardware (NVIDIA T4) as the median wall-clock time of 30 single-image forward passes at 640 × 640 resolution after five warm-up passes, in batch 1, FP32, from decoded RGB to CPU detections (excluding disk/decode). Parameter counts are taken from the model definitions.

5 Results

5.1 Overall Performance

Table 3 compares the three detectors on overall accuracy and inference cost. Faster R-CNN R50-FPN is the strongest detector overall, with $\text{mAP}_{50\text{-}95} = 0.333$ and $\text{mAP}_{50} = 0.483$. YOLOv8s follows closely at $\text{mAP}_{50\text{-}95} = 0.313$ and $\text{mAP}_{50} = 0.428$ (94% of Faster R-CNN's $\text{mAP}_{50\text{-}95}$), while RT-DETR-L trails at $\text{mAP}_{50\text{-}95} = 0.231$ and $\text{mAP}_{50} = 0.323$ in this nominally equal configuration.

Table 3: Overall detection performance and inference cost (median of 30 runs, batch 1, FP32, 640 px input, NVIDIA T4).

Detector	$\text{mAP}_{50\text{-}95}$	mAP_{50}	mAP_{75}	Latency (ms)
Faster R-CNN R50-FPN	0.333	0.483	0.354	46.4
YOLOv8s	0.313	0.428	0.356	24.0
RT-DETR-L	0.231	0.323	0.252	62.2

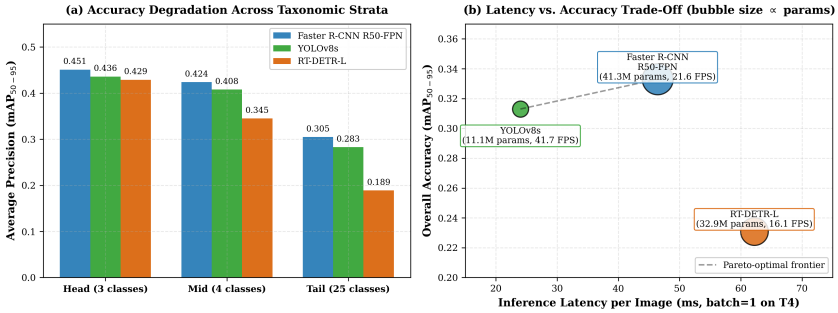
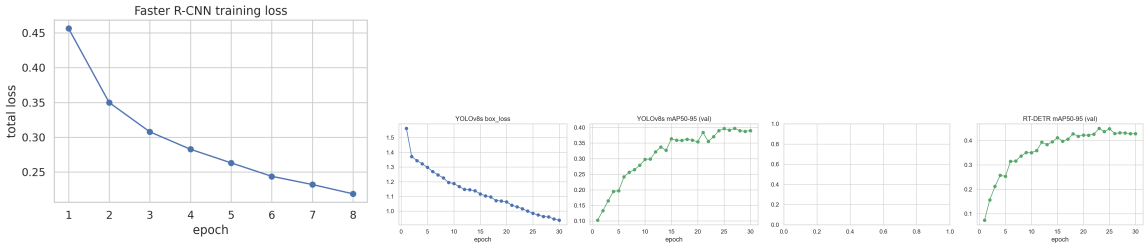


Figure 3: Accuracy–latency trade-off across the three detectors. Faster R-CNN leads in accuracy; YOLOv8s offers the best accuracy–cost balance; RT-DETR-L is neither the most accurate nor the fastest in this configuration.

Figure 3 visualizes the trade-off. YOLOv8s reaches 94% of Faster R-CNN's $\text{mAP}_{50\text{-}95}$ at roughly 52% of its latency and with approximately one quarter of the parameters, making it attractive for deployment on resource-constrained underwater vehicles. RT-DETR-L is the slowest (62.2 ms/image) and least accurate in this setup, so it is best viewed as the transformer-family baseline rather than a deployment recommendation from these runs.

5.2 Training Curves

Figure 4 shows the validation $\text{mAP}_{50\text{-}95}$ training curves for all three detectors over 30 epochs. Faster R-CNN and YOLOv8s both converge steadily, while RT-DETR-L exhibits a slower and noisier learning trajectory consistent with the known data-hunger and convergence challenges of end-to-end transformers.



(a) Faster R-CNN **(b)** YOLOv8s **(c)** RT-DETR-L

Figure 4: Validation $mAP_{50\text{-}95}$ training curves over 30 epochs for each detector (identical data split).

5.3 Per-Class Analysis

Table 4 reports per-class $AP_{50\text{-}95}$ for Faster R-CNN (the strongest detector), with representative values. Classes separate into clear tiers: easy classes (pyrosome, larvacean, hydroid, urchin, stony coral) achieve AP above 0.55, medium classes (anemone, sea fan, sponge, crab, octopus) lie in the 0.2–0.4 range, and hard tail classes (amphipod, bivalve, sea slug, sea squirt, barnacle) approach zero. The same ordering holds qualitatively for YOLOv8s and RT-DETR-L, confirming that per-class difficulty is driven primarily by the data distribution (annotation count and object size) rather than by architecture.

Table 4: Per-class $AP_{50\text{-}95}$ for Faster R-CNN R50-FPN (selected classes; full table in Appendix A).

Class	$AP_{50\text{-}95}$	Group
pyrosome	0.925	tail
larvacean	0.718	tail
hydroid	0.653	tail
urchin	0.592	head
stony coral	0.586	mid
soft coral	0.563	tail
crab	0.517	tail
bony fish	0.464	head
barnacle	0.454	tail
anemone	0.309	mid
sea fan	0.405	mid
sponge	0.397	mid
brittle star	0.297	head
sea cucumber	0.268	tail
benthic worm	0.001	tail
amphipod	0.000	tail
bivalve	0.000	tail
sea slug	0.000	tail
sea squirt	0.000	tail

Table 7 aggregates performance by head/mid/tail grouping. All three detectors show a consistent degradation from head to tail: head $AP_{50\text{-}95}$ ranges from 0.43–0.45, mid from 0.35–0.42, and tail from 0.19–0.30. This gradient is inherited directly from the long-tailed annotation distribution shown in Figure 1.

5.4 Object-Size Analysis

Table 5 reports size-stratified $AP_{50\text{-}95}$ (small/medium/large). Large objects are detected with moderate accuracy (0.25–0.38), medium objects poorly (0.06–0.08), and small objects effectively not at all (0.00–0.01). This reflects the 640×640 input resolution, under which many small

marine invertebrates occupy only a few pixels. Addressing small-object detection would require higher-resolution inputs, test-time tiling, or dedicated small-object augmentation — directions left for future work.

Table 5: Size-stratified $AP_{50\text{-}95}$ by detector at 640 px input.

Detector	Small	Medium	Large
Faster R-CNN R50-FPN	0.007	0.059	0.377
YOLOv8s	0.000	0.063	0.350
RT-DETR-L	0.000	0.075	0.254

5.5 Inference Cost

YOLOv8s is the fastest detector at 24.0 ms per image, Faster R-CNN intermediate at 46.4 ms, and RT-DETR-L slowest at 62.2 ms. For on-platform deployment, the trade-off is clear: Faster R-CNN is the accuracy leader but costs 1.9× the latency of YOLOv8s; YOLOv8s reaches 94% of Faster R-CNN's $mAP_{50\text{-}95}$ at roughly half the latency with far fewer parameters and fits comfortably on a 15 GiB GPU alongside the rest of a vehicle software stack. Memory constraints are significant: both Ultralytics runs used mixed precision with batch 4, while Faster R-CNN used batch 2 in single precision; larger batches or higher resolutions would exceed a single commodity GPU.

5.6 Qualitative Results

Figure 5 presents matched qualitative detection panels across the three architectures on representative validation frames at a score threshold of 0.25, with ground-truth boxes overlaid. Panels are drawn from the easiest classes (urchin, pyrosome) and the hardest (sea slug, amphipod). Detections on easy, large, frequent classes are consistent across all detectors; on rare, small, or camouflaged taxa, detections are sparse or absent even for the best model.

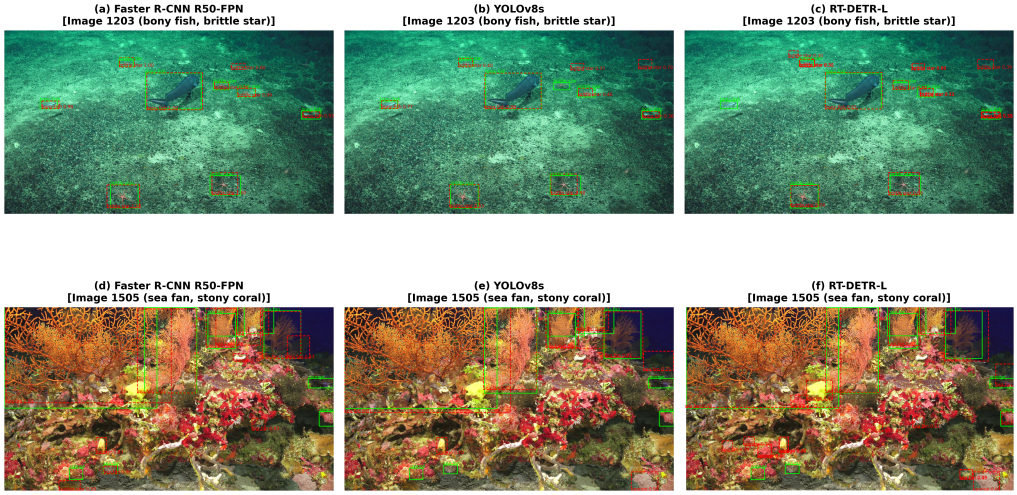


Figure 5: Qualitative detection panels for Faster R-CNN, YOLOv8s, and RT-DETR-L on representative validation frames (ground truth overlaid, score threshold 0.25).

5.7 Grad-CAM Analysis

Figure 6 shows high-resolution Grad-CAM visual attributions on the same scenes, comparing feature-activation patterns across deep network layers. The attention maps confirm that detectors

focus on salient foreground regions for easy classes, while activations are diffuse or absent for hard tail classes — consistent with the quantitative per-class results.

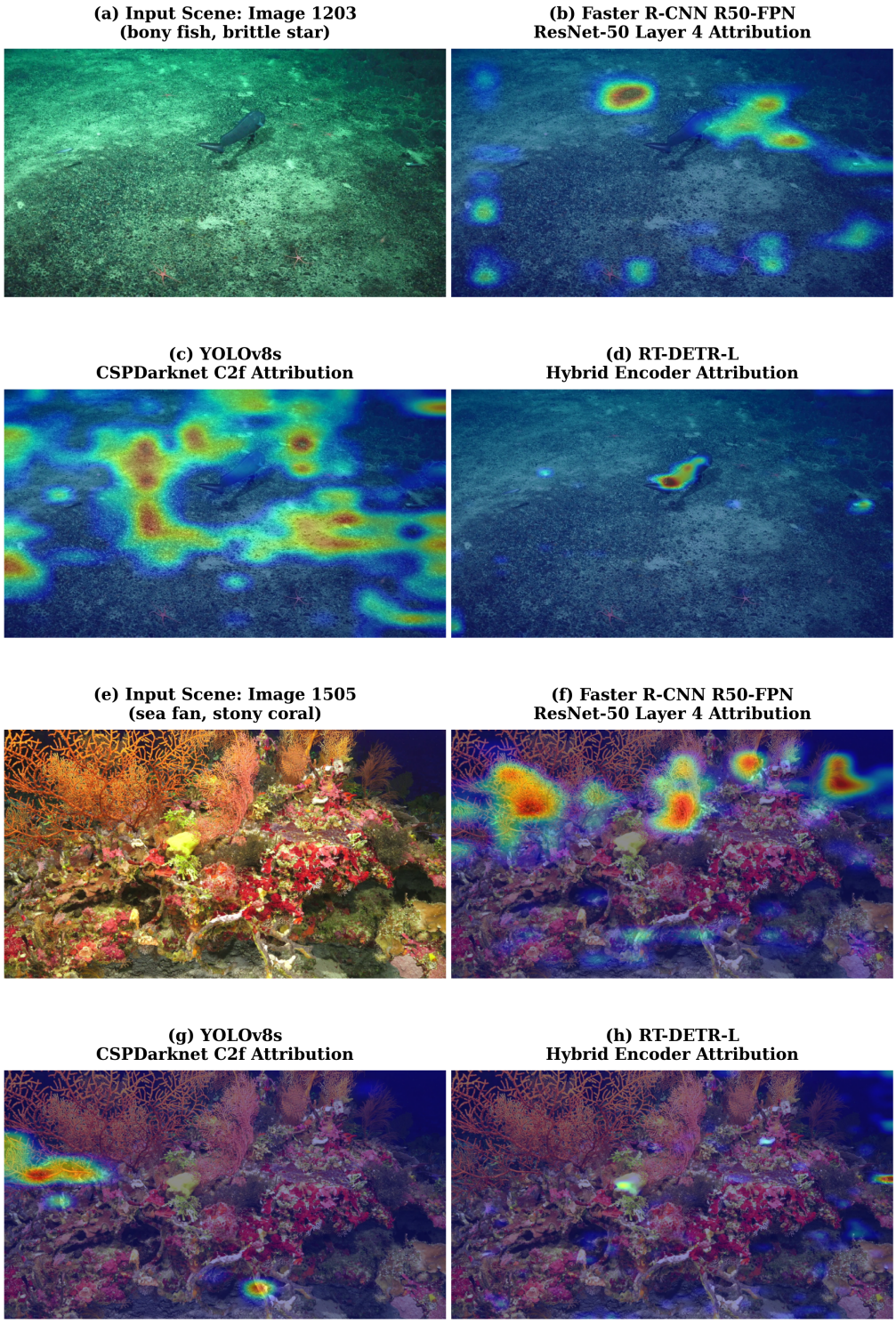


Figure 6: Grad-CAM visual attributions comparing feature activation across network layers for the three detectors.

6 Discussion

The observed ranking — two-stage > one-stage > transformer in this nominally equal 30-epoch, 640 px configuration — must be read with the configuration caveat of Section 4.2. All runs share the split, confidence threshold, detection cap, and nominal epoch/resolution budget, but framework-specific resizing rules, optimizers, augmentation, and checkpoint-selection rules differ; hence these numbers compare representative default training setups rather than a strictly controlled compute comparison. The head-to-tail degradation shared by all three runs is nevertheless inherited directly from the data distribution, and the per-class pattern is consistent across families.

For deployment on ROV/AUV platforms, YOLOv8s sits at the sweet spot of the accuracy–cost curve: near-Faster R-CNN accuracy at roughly half the latency and one quarter of the parameters, with modest memory requirements. Faster R-CNN is the accuracy ceiling in this configuration and remains preferable when inference latency is secondary to detection quality (e.g., offline post-processing of collected imagery). RT-DETR-L is neither the most accurate nor the fastest here and is best regarded as the transformer-family baseline; its relatively high memory footprint also makes it less attractive on single-GPU embedded platforms unless further optimized (distillation, quantization, smaller variant).

Which findings transfer? The head-to-tail degradation ordering should generalize to any long-tailed underwater dataset, since it reflects annotation frequency and object scale rather than architecture. The small-object blind spot at 640 px should also transfer across architectures and datasets. Exact mAP values, of course, are dataset- and protocol-specific.

7 Limitations

1. **Single split and seed.** Results are based on one stratified 90/10 split (seed 42); multi-seed variance is not reported.
2. **Validation-only evaluation.** No blind test-set results are reported; all numbers are on the validation split.
3. **Nominally equal, not identical, compute.** Framework-specific augmentation, optimizer, and checkpoint-selection rules differ across detectors.
4. **Single hardware.** Latency and memory measurements are specific to the NVIDIA T4 (15 GiB).
5. **Modest training budget.** All models train for 30 epochs; longer schedules, larger batches, or higher resolutions were not feasible within the memory and time constraints.
6. **No algorithmic long-tailed intervention.** Standard cross-entropy training is used throughout; class-balanced losses, resampling, or pseudo-labeling were not explored.

8 Conclusion and Future Work

This report benchmarks Faster R-CNN R50-FPN, YOLOv8s, and RT-DETR-L on FathomNet CLEF 2026 underwater imagery under a controlled protocol. Faster R-CNN achieves the best accuracy ($mAP_{50\text{-}95} = 0.333$), YOLOv8s offers the best accuracy–latency trade-off ($mAP_{50\text{-}95} = 0.313$ at 24 ms/image), and RT-DETR-L trails in this configuration ($mAP_{50\text{-}95} = 0.231$ at 62 ms/image). All detectors exhibit strong head-to-tail degradation driven by the long-tailed class distribution, and small objects are effectively undetected at 640 px resolution.

For practical AUV/ROV deployment, YOLOv8s is the recommended starting point; Faster R-CNN is the accuracy ceiling when latency is less critical. Future work includes: longer training

schedules and multi-seed evaluation; test-time tiling and higher-resolution inputs for small objects; long-tailed losses and resampling; model compression (quantization, distillation) for embedded deployment; and blind test-set submission to the FathomNet CLEF 2026 challenge.

A Full Per-Class AP Table

Table 6 reports the complete per-class $AP_{50\text{-}95}$ for Faster R-CNN R50-FPN across all 32 classes.

Table 6: Full per-class $AP_{50\text{-}95}$ for Faster R-CNN R50-FPN.

Class	$AP_{50\text{-}95}$
amphipod	0.000
anemone	0.309
barnacle	0.454
benthic worm	0.001
bivalve	0.000
black coral	0.246
bony fish	0.464
brittle star	0.297
calycophoran siphonophore	0.200
chiton	0.443
crab	0.517
feather star	0.354
hydroid	0.653
isopod	0.078
jelly	0.140
larvacean	0.718
octopus	0.426
physonect siphonophore	0.528
pyrosome	0.925
sea cucumber	0.268
sea fan	0.405
sea pen	0.222
sea slug	0.000
sea snail	0.167
sea squirt	0.000
sea star	0.267
shrimp	0.205
soft coral	0.563
sponge	0.397
squat lobster	0.238
stony coral	0.586
urchin	0.592

B Head/Mid/Tail and Size AP by Detector

Tables 7 and 8 summarize head/mid/tail and size-stratified results for all detectors.

Table 7: Head/mid/tail $AP_{50\text{-}95}$ by detector.

Detector	Head	Mid	Tail
Faster R-CNN R50-FPN	0.451	0.424	0.305
YOLOv8s	0.436	0.408	0.283
RT-DETR-L	0.429	0.345	0.189

Table 8: Size-stratified $AP_{50\text{-}95}$ by detector.

Detector	Small	Medium	Large
Faster R-CNN R50-FPN	0.007	0.059	0.377
YOLOv8s	0.000	0.063	0.350
RT-DETR-L	0.000	0.075	0.254

C Hardware and Software

Experiments were run on Kaggle using an NVIDIA T4 GPU with 15 GiB memory. The software stack comprises Python 3.9+, PyTorch [15], torchvision, `pycocotools` [5], Albumentations, OpenCV, and Ultralytics [10]. Code is organized under `src/` (data loading, detector wrappers, training and evaluation utilities) with experiment configs in `configs/` and Kaggle training notebooks under `notebooks/kaggle/`. All runs share split `split_fingerprint = d611d16e1c9c0562`, score threshold 0.001, and max detections 300.

References

[1] K. Katija et al., "FathomNet: A global image database for enabling artificial intelligence in the ocean," *Scientific Reports*, vol. 12, no. 1, p. 15914, 2022. https://doi.org/10.1038/s41598-022-19939-2

[2] FathomNet CLEF 2026 Organizers, "FathomNet CLEF 2026: ImageCLEF LifeCLEF track on underwater object detection," 2026. https://www.imageclef.org/

[3] O. Boulais et al., "FathomNet: An underwater image training database for ocean exploration and discovery," *arXiv preprint arXiv:2007.00114*, 2020.

[4] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, pp. 740–755, 2014.

[5] COCO Dataset Organizers, "pycocotools: Official COCO API." https://github.com/cocodataset/cocoapi

[6] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *NeurIPS*, vol. 28, pp. 91–99, 2015.

[7] T.-Y. Lin et al., "Feature Pyramid Networks for Object Detection," in *CVPR*, pp. 936–944, 2017.

[8] K. He et al., "Mask R-CNN," in *ICCV*, pp. 2980–2988, 2017.

[9] Ultralytics, "YOLOv8," 2023. https://github.com/ultralytics/ultralytics

[10] Ultralytics, "Ultralytics Python Package," 2023. https://github.com/ultralytics/ultralytics

[11] N. Carion et al., "End-to-End Object Detection with Transformers," in *ECCV*, pp. 213–229, 2020.

[12] X. Zhu et al., "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *ICLR*, 2021.

[13] Y. Zhao et al., "DETRs Beat YOLOs on Real-time Object Detection," in *CVPR*, pp. 16965–16974, 2024.

[14] L. Chen et al., "Underwater Object Detection in the Era of Artificial Intelligence: Current, Challenge, and Future," *arXiv preprint arXiv:2410.05577*, 2024.

[15] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *NeurIPS*, vol. 32, pp. 8024–8035, 2019.

[16] Kaggle, "Kaggle: Your Machine Learning and Data Science Community," 2026. https://www.kaggle.com/